

CSE211s Introduction to Embedded Systems

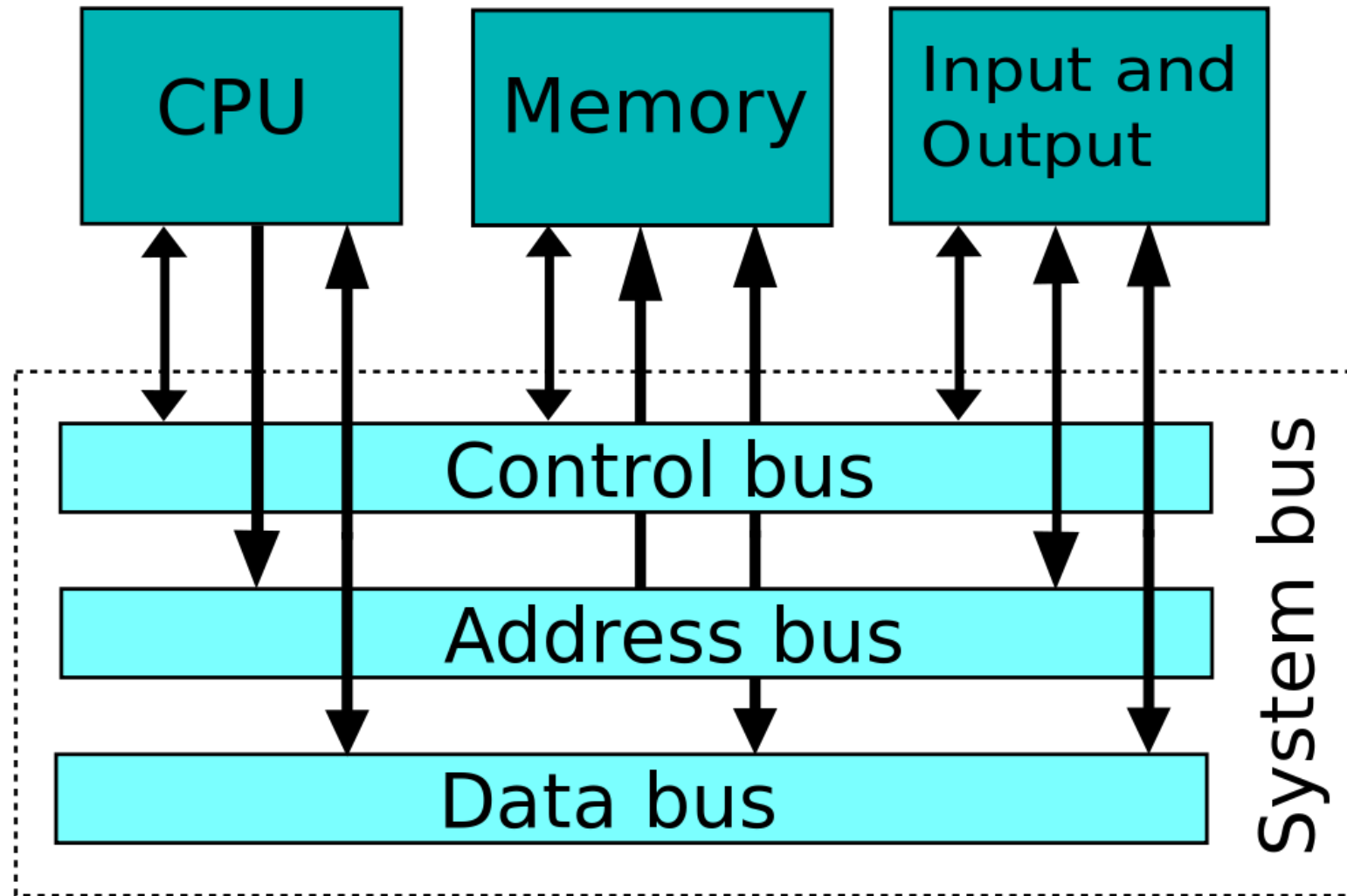
Spring 2026

CPU + Memory + IO

Topics

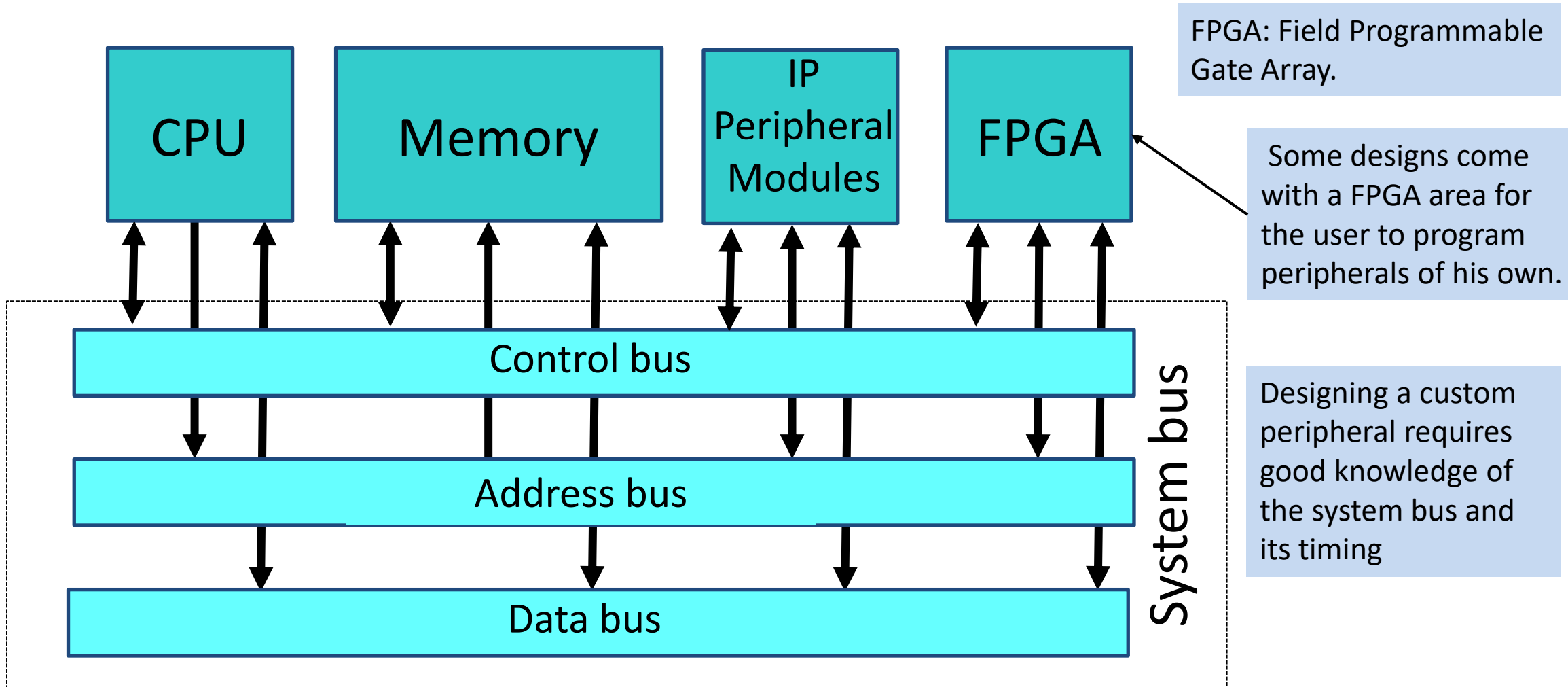
- **Components of a Digital Computer**
 - CPU
 - Memory
 - Input/Output (IO)
- **Computer Architecture**
 - Von Neumann and Harvard Architectures
- **Instruction Set Architecture**
 - CISC and RISC architectures
 - ARM Cortex-M Programming Model
- **Memory**
 - Memory Types: ROM / RAM
 - Memory Organization and Addressing
 - Memory Map

Basic Components of a Digital Computer

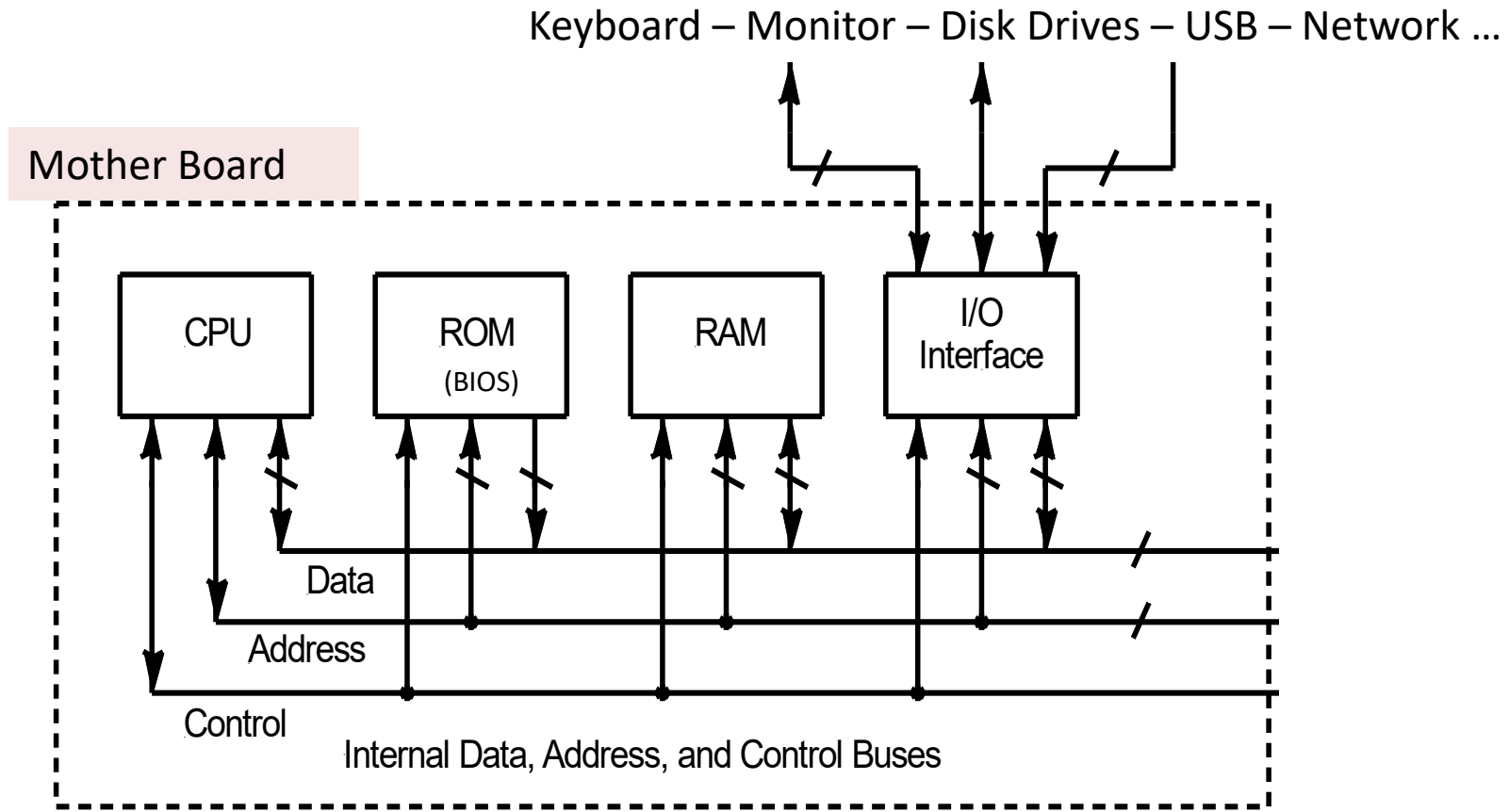


Source: Wikipedia.com

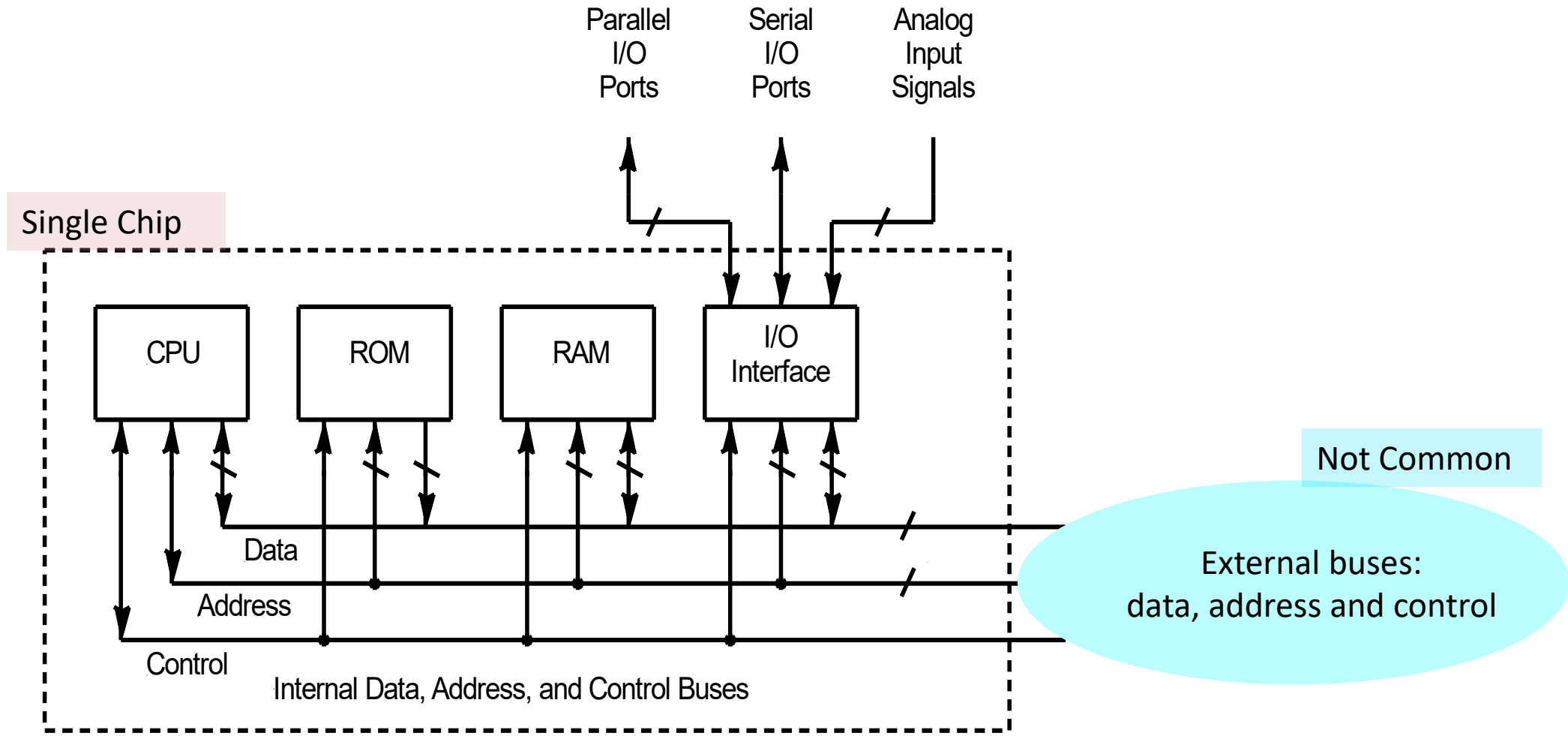
Typical Microcontroller Components



Basic Components of a Digital Computer



Typical Microcontroller Components



Function of the CPU

The CPU performs basic arithmetic, logic, controlling, and input/output (I/O) operations specified by the instructions.

The CPU

- 1- fetches
- 2- decodes
- 3- executes program instructions

To be able to do its function, a CPU includes:

1. **The ALU**

Performs arithmetic and logic operations.

2. **Registers**

- General purpose registers to hold operands to operate on.
- Special purpose registers such as the Program Counter and the Stack Pointer

3. **The control unit**

- Fetches instructions from memory, decodes, and "executes" them.
- Handles Exceptions and interrupts.

The CPU may contain Memory Control Unit (MMU) and Cache Memory.

CPU Properties

Clock Speed (MHz)

Higher frequency → more instructions per second, but also → higher power consumption
Clock speed alone does *not* equal performance. Architecture and pipeline design matter too.
Example: 84 MHz Cortex-M4 may outperform a 100 MHz older architecture.

Performance Metrics

- Million Instruction Per Second (MIPS): Gives realistic performance comparison across MCUs.
- Cycles per instruction (CPI): Lower CPI = better efficiency.

Word Size (8/16/32-bit)

- 8-bit → simple, very low cost/power
- 16-bit → mid-range
- 32-bit (e.g., ARM Cortex-M) → modern standard

For most modern embedded systems, **32-bit is preferred**.

FPU (Floating Point Unit)

Hardware floating-point support:

Important in control systems, filtering, sensor fusion, DSP and robotics, Otherwise, software floating point is slow.

CPU Properties

DSP Extensions

- MAC (multiply-accumulate)
- SIMD instructions
- Saturation arithmetic

Interrupt System

- Nested vectored interrupt controller (NVIC)
- Interrupt latency
- Number of interrupt lines
- Priority levels

Critical for real-time systems, motor control, communications

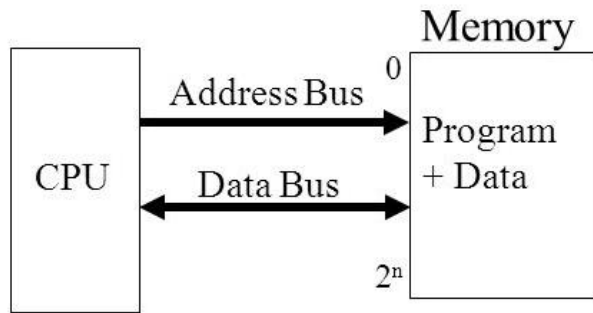
Low-Power Features (CPU-Level)

- Sleep modes
- Deep sleep
- Stop/standby modes
- Wake-up latency

Critical for battery-powered systems

Security Features

Von Neumann vs Harvard Computer Architectures

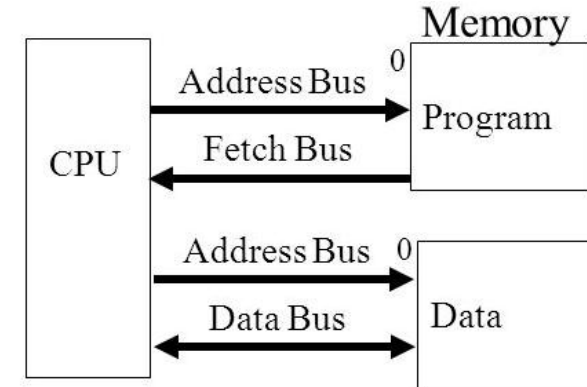


Von Neumann architecture (1945)

Has one set of address and data buses for reading and writing to memory.

A data read/write cannot occur at the same time with an instruction fetch since they share a common bus.

This is referred to as the Von Neumann bottleneck, which often limits the performance of the corresponding system.



Harvard architecture (1939)

Has two separate memory address and data buses, one for reading and writing data to memory, and another for fetching instructions.

It is most often found in microcontrollers used in embedded systems, where the programs are stored in **ROM**, and data is stored in static **RAM**

The design of a von Neumann architecture machine is simpler than in a Harvard architecture machine.

Example Computer Architectures

1. Von Neumann architecture:

ARM Cortex-M0 and M0+ processors.

Motorola HC12 processor

2. Harvard architecture:

ARM Cortex-M3 and M4 processors:

- Instructions are fetched from flash ROM using the ICode bus
- Data are exchanged with RAM and I/O via the system bus.
- Some internal peripherals, like the NVIC, communicate directly with the processor via the private peripheral bus (PPB) to provide fast interrupt handling.

The Instruction Set Architecture (ISA)

- An instruction set is a group of commands recognized by a CPU in machine language.
- An Instruction Set Architecture (ISA) defines:
 - The supported (recognized) instructions
 - The data types (byte, word, halfword)
 - CPU registers
 - Addressing modes
 - Input/output model

CISC and RISC CPU Architectures

Researchers at Berkeley in the late 70s found that most programs used only a small subset of the instruction set.

If they reduce the number of predefined instructions—cutting out complex, hard to implement, and little used instructions —the remaining simple instructions would:

1. *run faster,*
2. *require less CPU power*
3. *take up less memory.*

A computer designed with such architecture is called reduced instruction set computer (RISC) as opposed to complex instruction set computer (CISC).

CISC architecture reduces the number of instructions per program, while RISC emphasizes simple instructions executed quickly.

CISC vs RISC Architectures

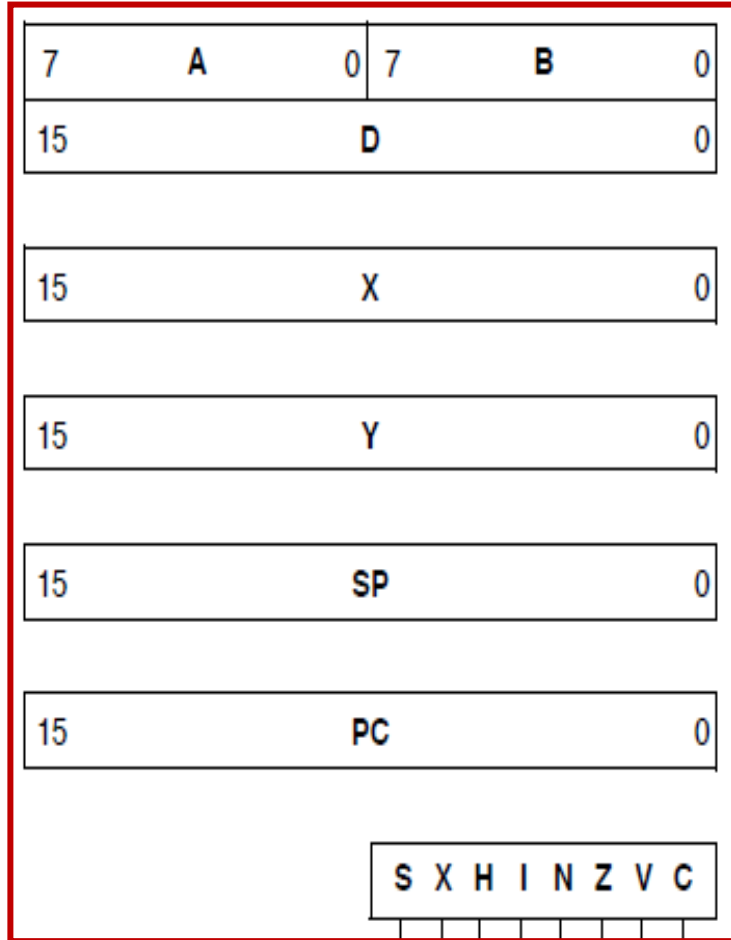
CISC	RISC
Has instructions for arithmetic & logical operations with operands in memory (Definitely through some invisible CPU registers)	Memory is referenced only through LOAD & STORE instructions. (<i>Load-Store architecture</i>)
Variable length instructions, often longer.	Usually, fixed length instructions, often shorter
Instructions take multiple cycles.	Instructions complete in 1 cycle. (Some in 2 cycles)
To do an operation on an operand in memory	
A single instruction does the following: 1. Load the operand from memory to an <i>invisible</i> CPU register 2. Perform the operation on the register 3. Store the result back in memory	Use three instructions: 1. A LOAD instruction to get the operand from memory to a CPU register 2. An instruction to perform the operation on the CPU register 3. A STORE instruction to write the result from register back in memory
➤ Complex instruction → Complex CPU circuitry → Extra power	➤ Simple instructions → Simple CPU circuitry → Less power ➤ But more memory references → <i>slower</i> performance (<i>overcome by improved pipelining</i>)
—Motorola 6800, 6809, 68000 —Freescale HC11, and HC12 —intel x86 family, i5 i7, and 8051-family	—ARM, MIPS, RISC V —Atmel AVR, PowerPC —intel i860 and i960

CPU Programming Model

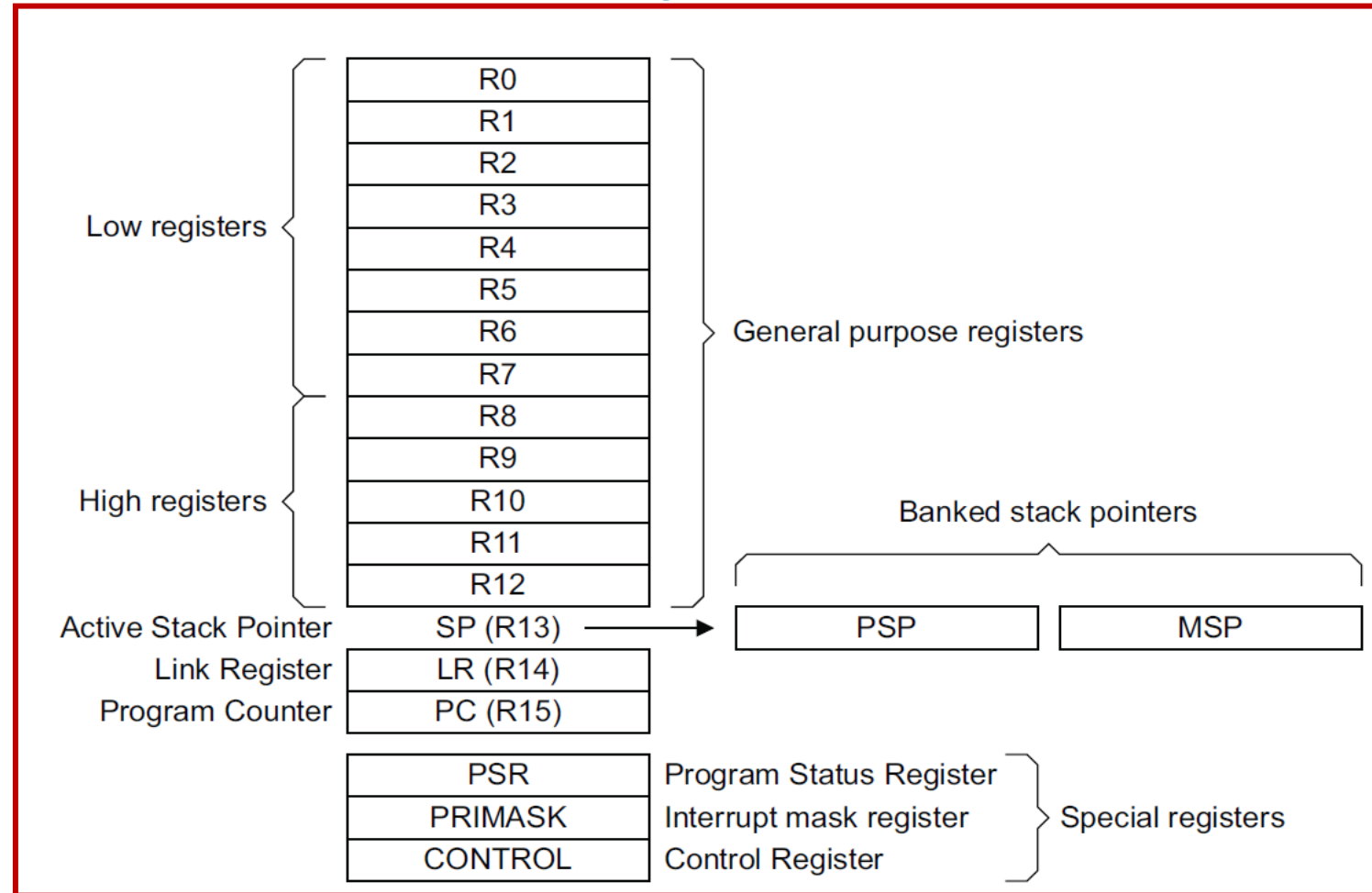
CISC/RISC Examples

The programming (programmer's) model is the group of registers that are visible by the programmer and the function of each.

HC12 Core Registers



ARM Cortex-M Registers



CISC CPUs usually have fewer registers compared to RISC ones

Memory Types

- **Volatile:** Random access memory RAM (Read & write).
- **Nonvolatile (NVM):** Retains its contents without power supply.

Types of NVM:

- ROM:** Read only memory. This term is usually used to refer to devices programmed by the manufacturer.
- PROM:** Programmable read only memory. Programmed once by the user with special hardware.
- EPROM:** Erasable programmable read only memory. Erased as a whole with ultra violet light, and reprogrammed with special hardware.

- EEPROM:** Like EPROM but it is erasable and reprogrammable by electrical signal, (no UV).
It can be reprogrammed on board one word at a time. *Holds constants.*
- FLASH:** A type of EEPROM that can be erased and programmed in sectors (blocks). *Hold the code, (program and constants)*

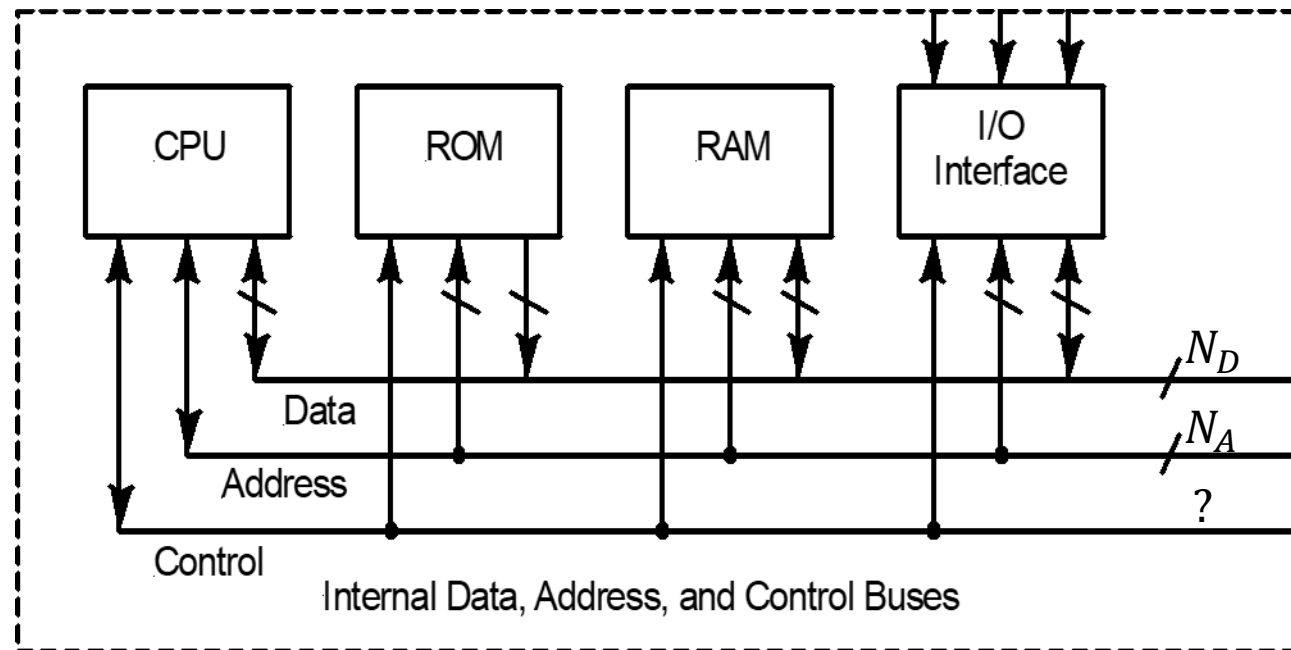
➤ New technologies for NVM are still emerging; suitable for big data applications.

Memory Organization

- Memory is usually byte-organized (byte-addressable)
 - ↳ ***Each byte in memory has a distinct address.***

The memory is interfaced to the CPU through:

- Address bus (N_A bits)
- Data bus (N_D bits)
- Control signals (Read/Write, Wait ...)



Memory Size

The maximum addressable memory size is 2^{N_A} bytes

- The first-location address is 0
- The last-location address is $2^{N_A} - 1$

Examples:

- 10-bit address bits can address $2^{N_A} = 2^{10} = 1024 = 1\text{k}$ of memory.
- A 64k memory is $64 \times 1\text{k} = 2^6 \times 2^{10} = 2^{16}$ needs 16-bit address.

A CPU can read/write one, two or four bytes in a single read/write in accordance with the data bus size N_D .

Example:

If the data bus is 4 bytes, then a read from address 0x1000 retrieves four bytes from memory locations

0x1000

0x1001

0x1002

0x1003

Memory Map

A **memory map** is a structured layout that shows how different types of memory and hardware resources are arranged and addressed within a computer system or microcontroller.

It tells you:

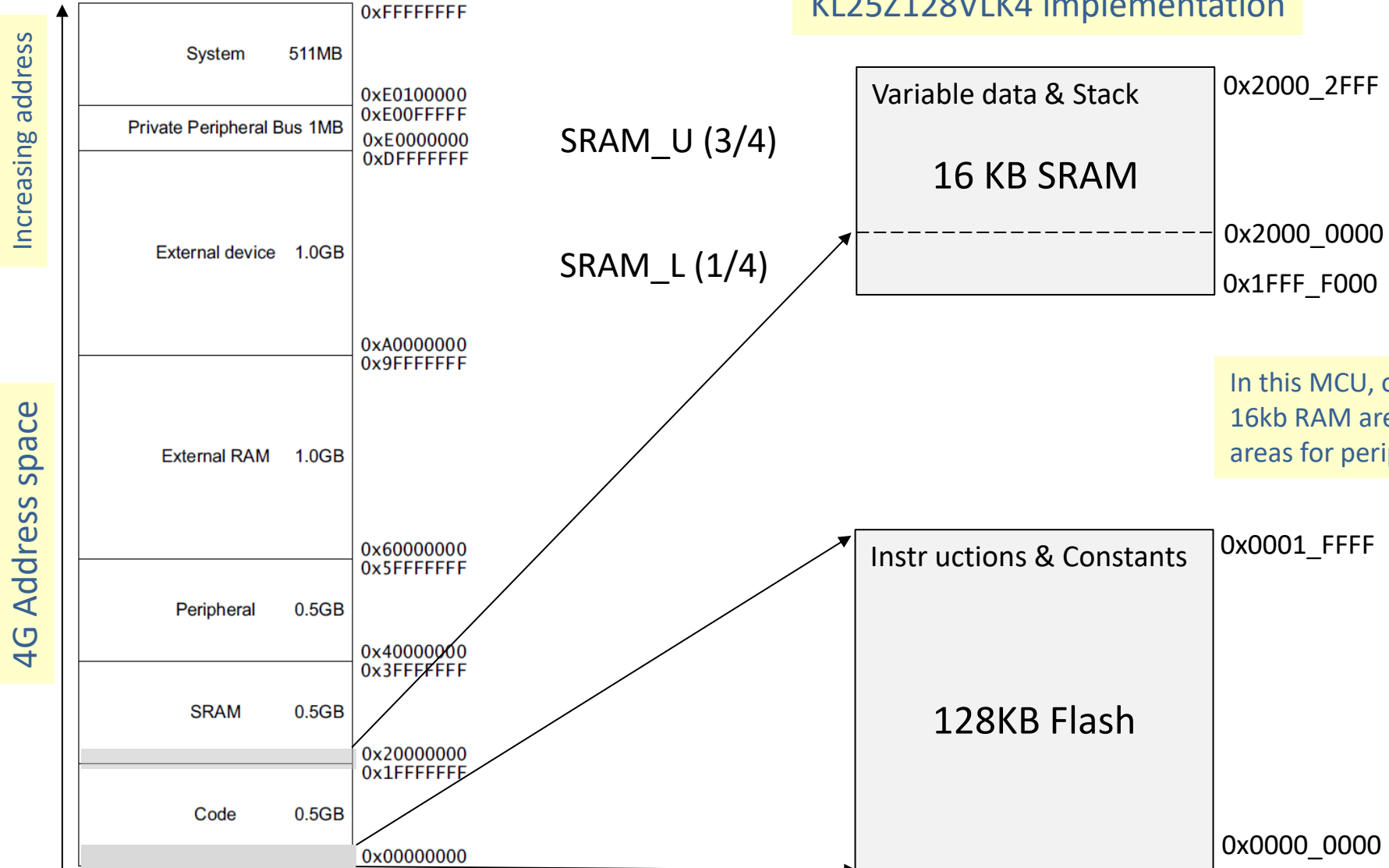
- What memory exists (Flash, RAM, EEPROM, peripherals, etc.)
- Where each memory region starts and ends (address ranges)
- What each region is used for

Memory Map

Example: ARM Cortex-M

KL25Z128VLK4 implementation

A memory map indicates how memory is laid out.



In this MCU, only 128kb flash and 16kb RAM are implemented plus the areas for peripherals and the system

32-bit address → 4G address space

Memory Map

Example: Motorola HC12

MC9S12C32 implementation

Note that in this diagram of the memory map, the lowest address is on the top and the address increases towards the bottom.

In the previous diagram of memory map of the ARM, the lowest address is in the bottom and the address increases towards the top.

64k Address space

Increasing address

\$0000	½ Kbytes I/O Interface Registers
\$01FF	
\$0200	½ Kbytes EEPROM
\$03FF	
\$0400	None
\$07FF	
\$0800	
\$0FFF	Relocatable 2 Kbytes RAM
\$8000	
\$FFFF	32 Kbytes Flash

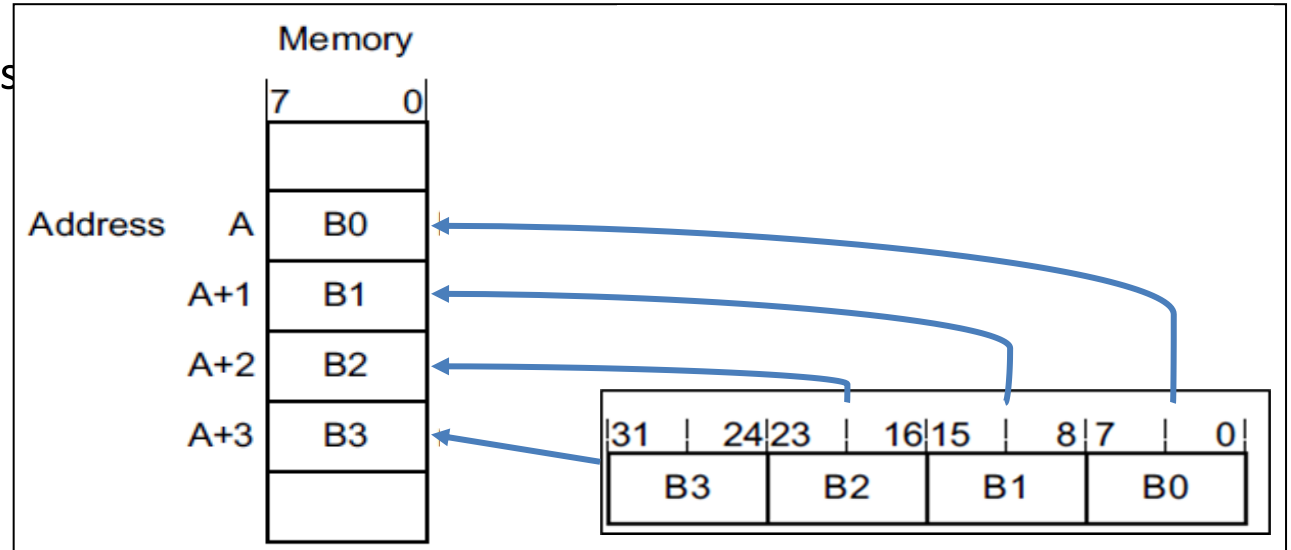
16-bit address

Endianness

For a multi-byte value, in what order are the bytes stored in memory?

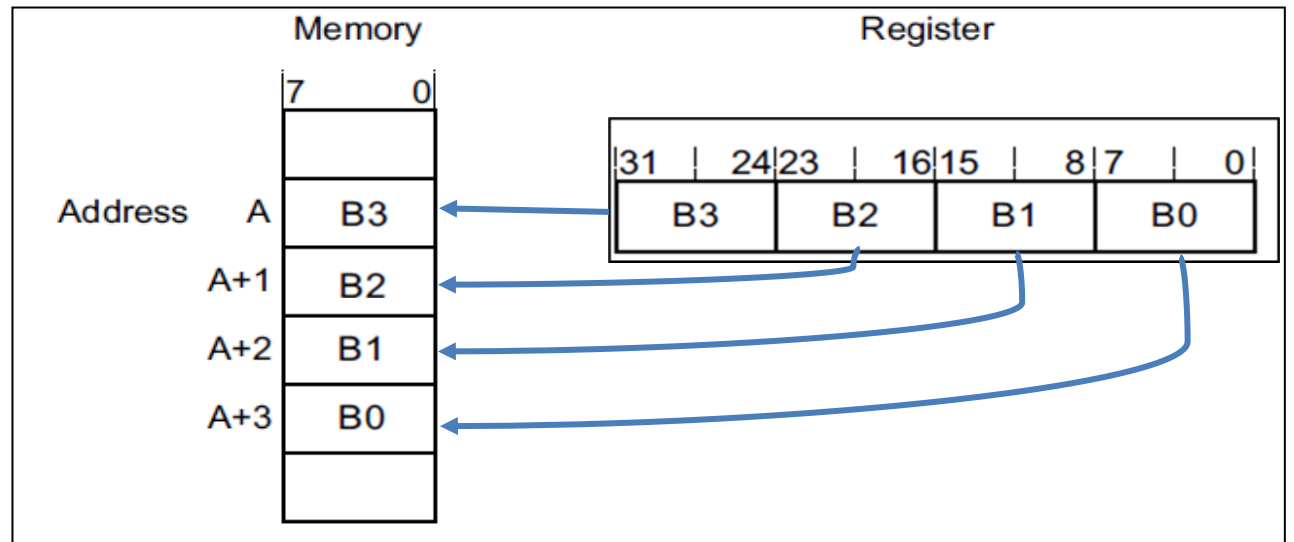
- **Little-Endian:**

Store the register in address A, start with least-significant byte



- **Big-Endian:**

Store the register in address A, start with most-significant byte



Input/Output (IO) Addressing

A microcontroller typically is connected to the real world through multiple ***IO peripheral modules*** that interface the CPU to switches, displays, analog sensors & actuators and communication links.

The CPU sees each ***IO peripheral module*** as a group of ***IO registers***.

The CPU uses these registers to

1. Exchange data with the *peripheral module*
2. Send configuration commands to the *peripheral module*
3. Read the status of the *peripheral module*

Like memory which needs addressees to its locations, the CPU uses addresses to IO registers.

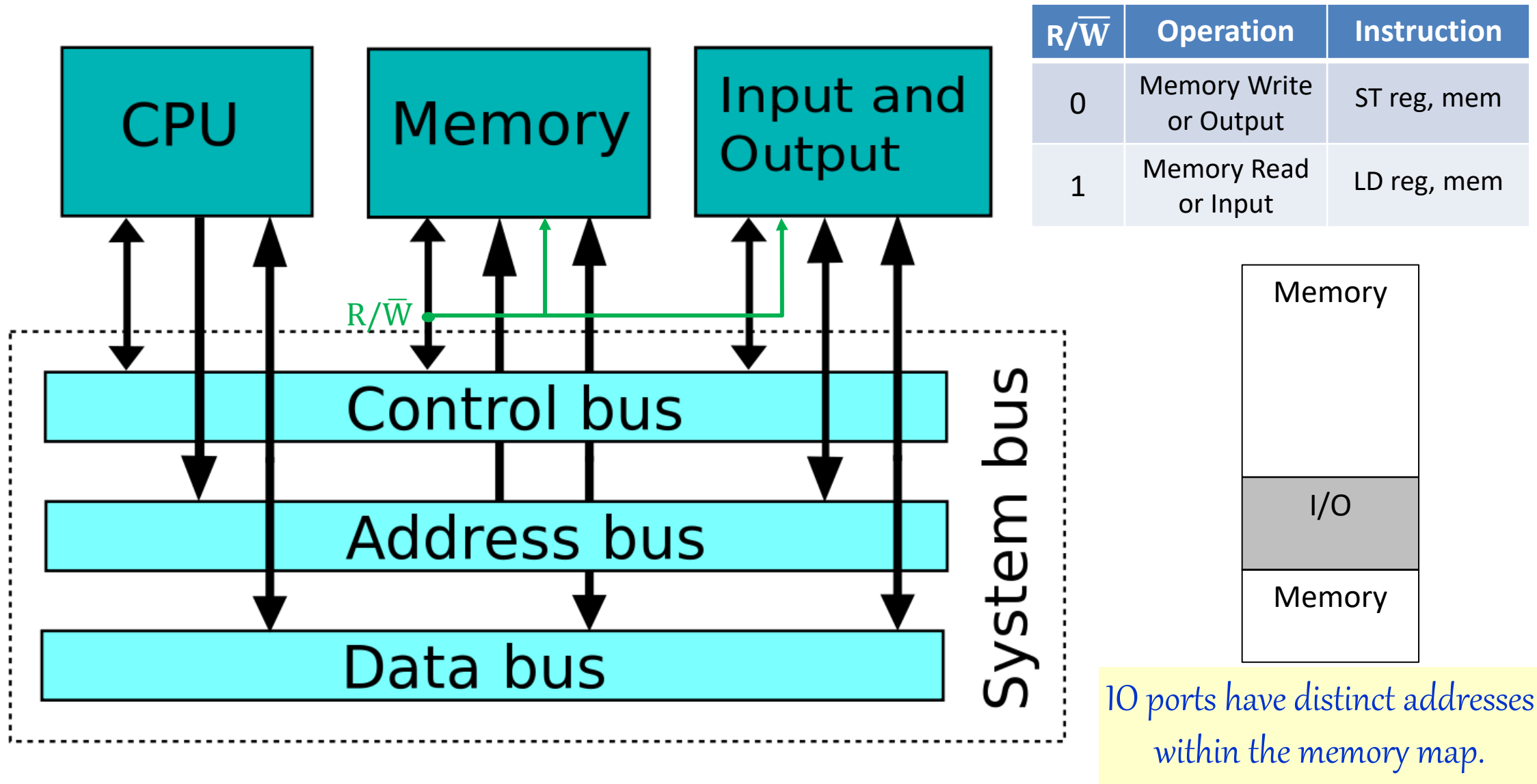
The application software uses these addresses for all its IO operations with the peripheral module.

Input/Output (IO) Addressing

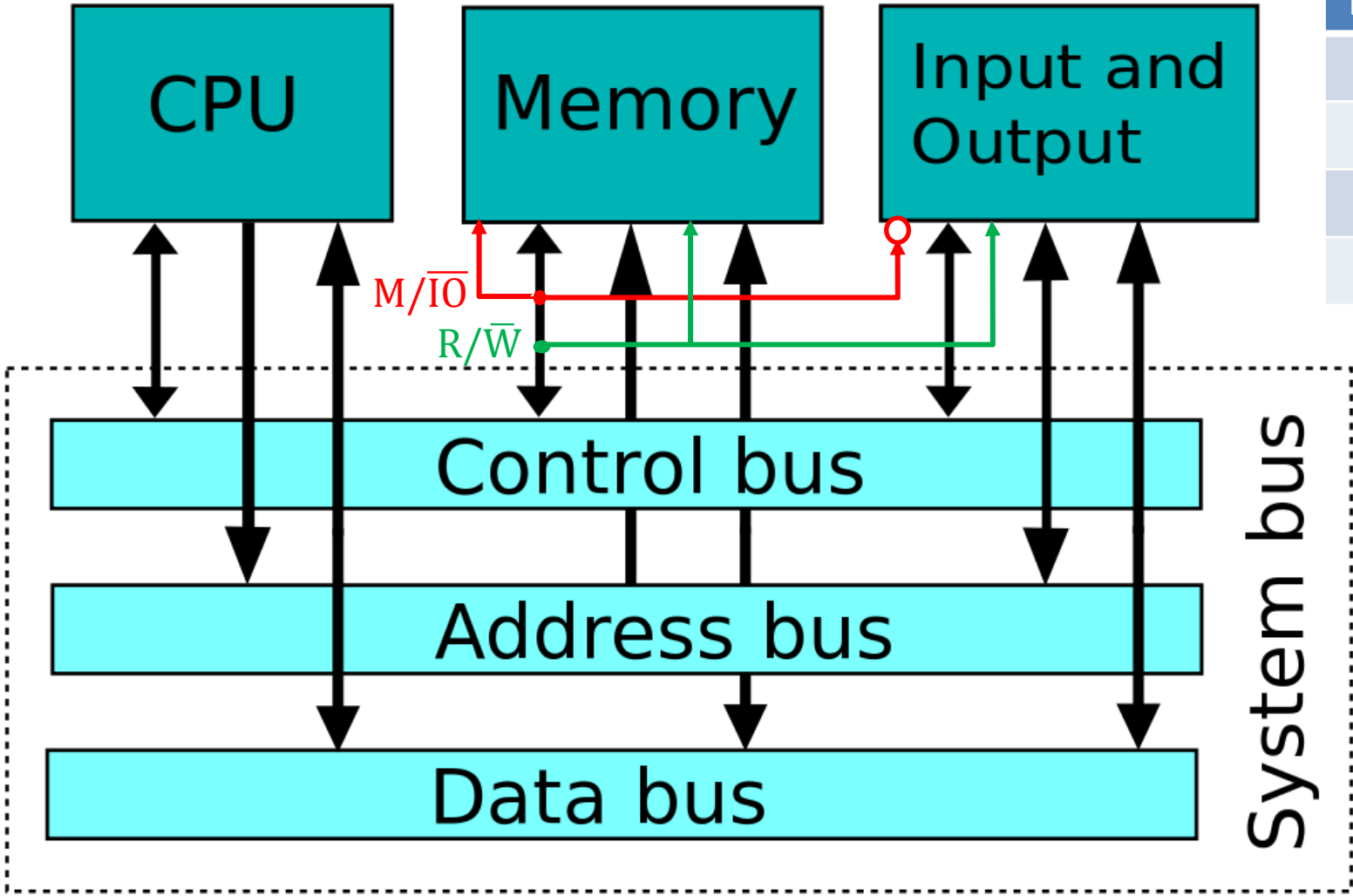
Memory-Mapped IO	Isolated (Port-Mapped) IO
➤ The CPU uses the same memory reference instructions, LOAD and STORE, to do IO operations.	➤ The CPU instruction set has distinct IN and OUT instructions to do IO operations.
➤ IO port addresses do not overlap with memory locations addresses. IO ports occupy exclusive area in the memory map.	➤ IO port addresses can overlap with memory addresses. An address can be shared by a memory location and an IO port.
➤ Used in ARM processors and most MCUs.	➤ Used in PCs and intel x86, i5 and i7 line of processors.
➤ The MemoryRead and MemoryWrite signals on the control bus are used to reference IO ports as well.	➤ The control bus has a signal ($\overline{\text{Mem/IO}}$) that differentiates I/O cycles from memory cycles; $\overline{\text{Mem/IO}} = 0$ when executing a memory ref. instruction $\overline{\text{Mem/IO}} = 1$ when executing an IN or OUT instruction

Extra Readings

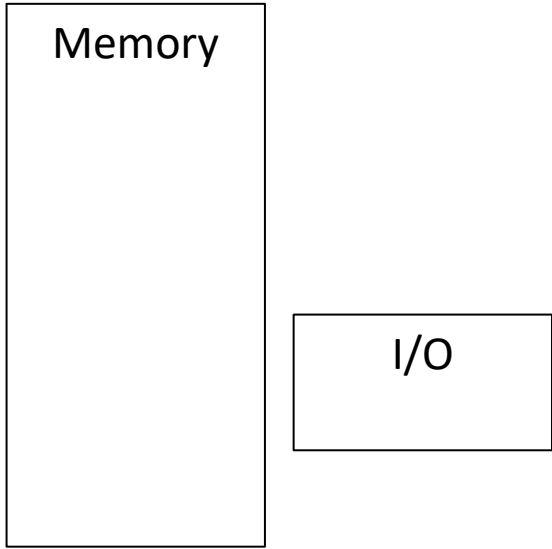
Memory-Mapped I/O



Isolated IO (Port-Mapped IO)



M/ $\overline{IO}$	R/ $\overline{W}$	Operation	Instruction
0	0	Output	OUT reg, port
0	1	Input	IN reg, port
1	0	Memory Write	ST reg, mem
1	1	Memory Read	LD reg, mem



IO ports addresses can overlap with memory locations in the memory map.

ARM Cortex Processors

Cortex-A (Application)

Supreme performance at optimal power

Cortex-M (Embedded)

Powering the most energy-efficient embedded devices

Cortex-R (Realtime)

Robust real-time performance

Ethos - NPUs

Highest performance for machine learning inference

Neoverse

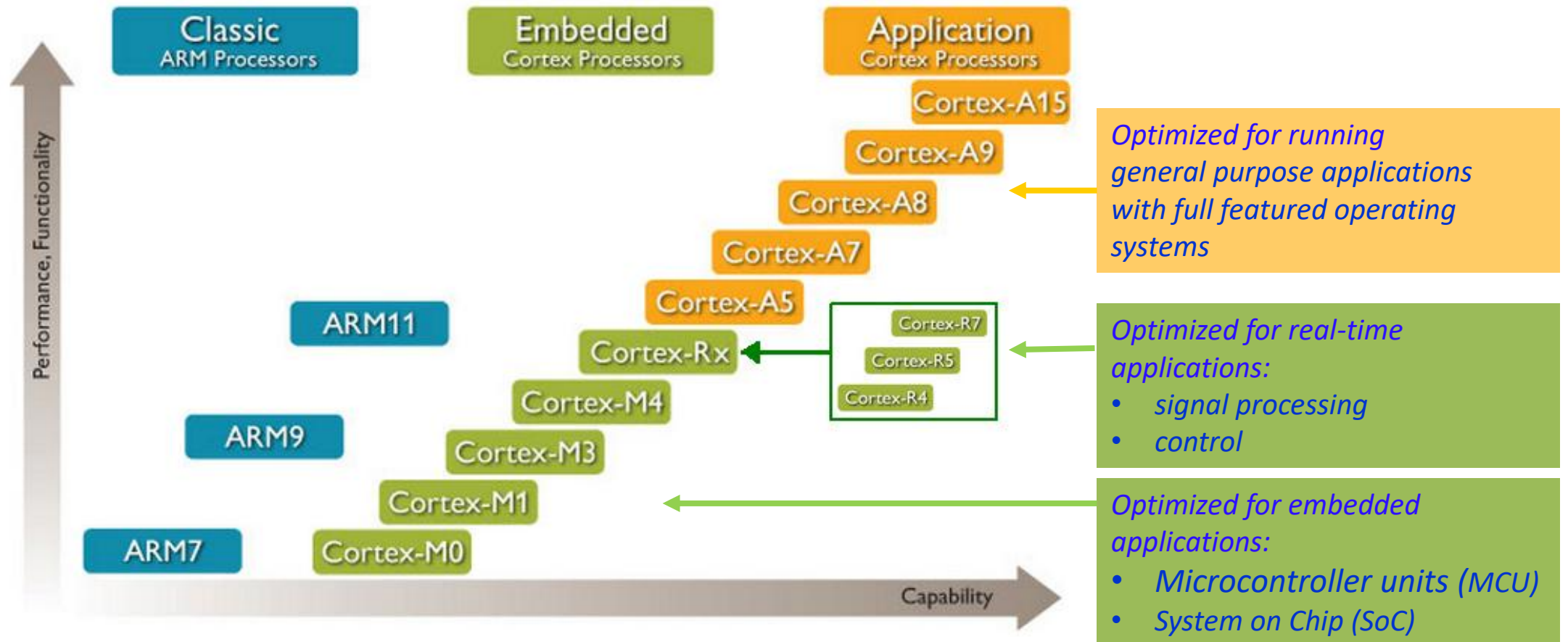
Scalable and flexible for cloud to edge infrastructure

SecurCore

Powerful solutions for physical security applications

ARM Cortex Processors

A



Microcontroller Families

Microcontrollers are grouped in families based on the CPU used.

↪ Microcontrollers within the same family differ by:

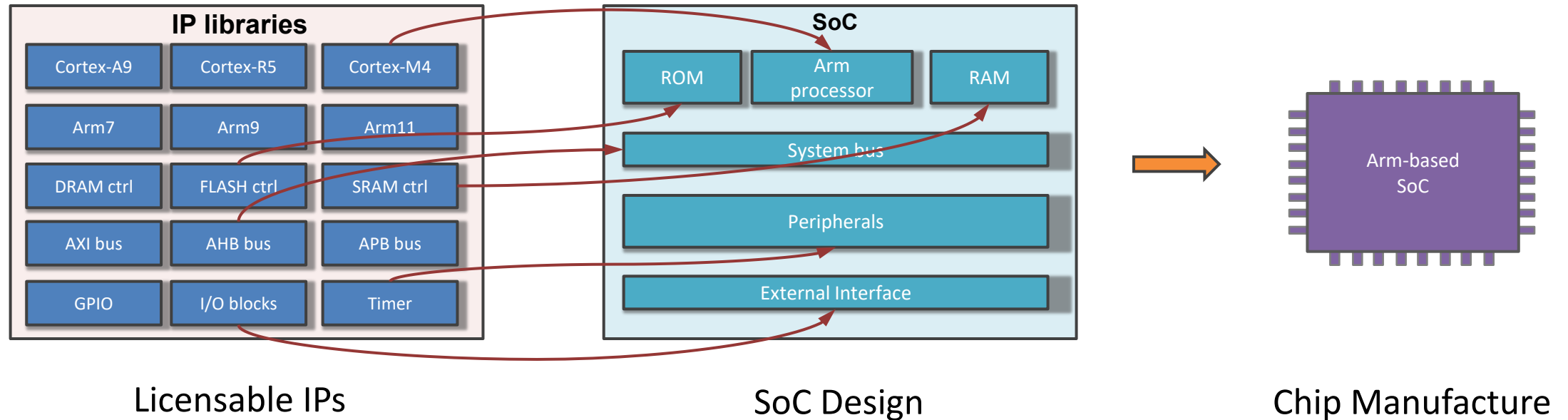
- The amount of memory (ROM and RAM)
- The types of I/O modules (peripherals)
- Packaging

ARM Cortex-M microcontrollers have similar memory maps:

- Flash ROM begins at address 0x00000000
- RAM begins at 0x20000000
- Peripheral I/O space starts from 0xE0000000 to 0xE00FFFFF

How to Design an Arm-based SoC

1. Select a set of IP cores from Arm and/or other third-party IP vendors
2. Integrate IP cores into a single-chip design
3. Give design to semiconductor foundries for chip fabrication



Arm Cortex-M Series Family

Processor	Arm Architecture	Core Architecture	Thumb	Thumb-2	Hardware Multiply	Hardware Divide	Saturated Math	DSP Extensions	Floating Point
Cortex-M0	Armv6-M	Von Neumann	Most	Subset	1 or 32 cycle	No	No	No	No
Cortex-M0+	Armv6-M	Von Neumann	Most	Subset	1 or 32 cycle	No	No	No	No
Cortex-M3	Armv7-M	Harvard	Entire	Entire	1 cycle	Yes	Yes	No	No
Cortex-M4	Armv7E-M	Harvard	Entire	Entire	1 cycle	Yes	Yes	Yes	Optional
Cortex-M7	Armv7E-M	Harvard	Entire	Entire	1 cycle	Yes	Yes	Yes	Optional

Questions

1. What is the difference between RAM and ROM in embedded systems?
2. Why is RAM size critical in embedded systems?
3. What happens if the stack grows beyond available RAM?
4. Why is understanding CPU and memory important for embedded programming in C?